

# Advanced FFT Algorithms and Applications

## Signals and Linear Systems

---

Ashrafur Rahman  
Lecturer, CSE, BUET

[ashrafur@cse.buet.ac.bd](mailto:ashrafur@cse.buet.ac.bd)

### Lecture 3 — the FFT

- Radix-2 DIT: split  $x[n]$  into evens and odds, combine with the butterfly

$$X[k] = G[k] + W_N^k H[k]$$

$$X[k + N/2] = G[k] - W_N^k H[k]$$

- Cost drops from  $N^2$  to  $N \log_2 N$ .
- But it needs  $N = 2^m$ . Otherwise we zero-pad — which changes a periodic signal.

### Lecture 2 — filtering with the DFT

- $x \circledast h \leftrightarrow X[k]H[k]$ ; pad to  $N \geq L + M - 1$  and circular = linear.
- The recipe: pad, transform, multiply, invert.
- Honest accounting then: each DFT cost  $N^2$ , so the recipe lost to the direct  $L \cdot M$  sum.

### Today's two questions

1. Can we have an FFT for any  $N$  — powers of 3, mixed factors, even primes — without padding?
2. With the FFT in hand, how fast does the filtering recipe become?

- **Advanced FFT Algorithms**

- Radix-2 and Radix-3 DIT: the tabular (matrix) interpretation
- General Cooley–Tukey factorization  $N = N_1 N_2$  and Bailey’s four-step FFT

- **Application: Fast Convolution**

- The Lecture-2 filtering recipe, now with the FFT
- Complexity:  $O(N \log N)$  versus  $O(LM)$

- **FFT for Prime  $N$ : Bluestein’s Algorithm**

- The chirp-z identity: a DFT is a linear convolution
- Padding the chirp without breaking the wrap

## Advanced Matrix Interpretations of the FFT

---

- Previously, we derived Radix-2 DIT purely algebraically.
- We divided the input into even and odd indices and calculated their DFTs  $G[k]$  and  $H[k]$ .
- Then we combined them using the butterfly operation:

$$X[k] = G[k] + W_N^k H[k]$$

$$X[k + N/2] = G[k] - W_N^k H[k]$$

- We can think about this in a different, more visual way—one that leads to an elegant generalization.
- **Key Idea:** Reshape the 1D input  $x[n]$  into a 2D matrix, perform DFTs along rows and columns.

- Let us arrange  $x[n]$  into a  $2 \times N/2$  matrix in **column-major** order:

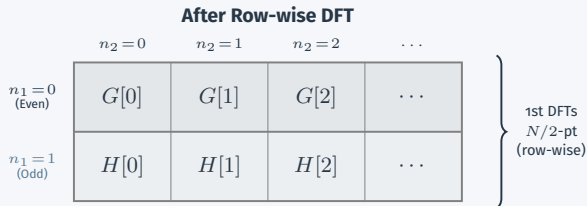
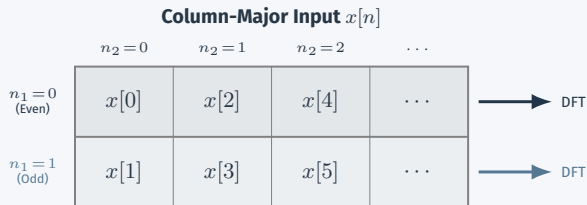
$$n = 2n_2 + n_1$$

Where,

$n_1 = \text{row index} \in \{0, 1\}$

$n_2 = \text{column index} \in \{0, 1, \dots, N/2 - 1\}$

- Row 0** ( $n_1 = 0$ ): Even indices
- Row 1** ( $n_1 = 1$ ): Odd indices
- 1st DFT:** Then perform  $N_2$ -point DFTs along each row ( $\rightarrow$ ).
  - Row 0  $\rightarrow G[k]$
  - Row 1  $\rightarrow H[k]$



- After row DFTs, multiply row  $n_1$  by twiddle factors  $W_N^{n_1 k}$ .

- Row 0:  $\times W_N^0 = 1$
- Row 1:  $\times W_N^k$

- 2nd DFT:** Perform 2-point DFT down each column ( $\downarrow$ ).
- This is exactly the butterfly!

$$X[k] = G[k] + W_N^k H[k]$$

$$X[k + \frac{N}{2}] = G[k] - W_N^k H[k]$$

- We obtain the DFT of the original sequence  $x[n]$ !
- Row 0 gives  $X[0 \dots N/2 - 1]$ ,  
row 1 gives  $X[N/2 \dots N - 1]$ .
- But the output occurs in **row-major** order.

## After Twiddle Factors

|                     | $n_2 = 0$   | $n_2 = 1$   | $n_2 = 2$   | $\dots$ |
|---------------------|-------------|-------------|-------------|---------|
| $n_1 = 0$<br>(Even) | $G[0]$      | $G[1]$      | $G[2]$      | $\dots$ |
| $n_1 = 1$<br>(Odd)  | $H[0]W_N^0$ | $H[1]W_N^1$ | $H[2]W_N^2$ | $\dots$ |

$\downarrow$   
DFT

$\downarrow$   
DFT

$\downarrow$   
DFT

$\downarrow$   
DFT

## After Column-wise DFT

|                     | $n_2 = 0$ | $n_2 = 1$    | $n_2 = 2$    | $\dots$ |
|---------------------|-----------|--------------|--------------|---------|
| $n_1 = 0$<br>(Even) | $X[0]$    | $X[1]$       | $X[2]$       | $\dots$ |
| $n_1 = 1$<br>(Odd)  | $X[N/2]$  | $X[1 + N/2]$ | $X[2 + N/2]$ | $\dots$ |

## Radix-2 DIT: Reading the Output Row-Major

- We want  $X[k]$  in the correct order. We want the transpose matrix.
- Mathematically, for

$$k_1 = \text{row index} \in \{0, 1\}$$

$$k_2 = \text{column index} \in \{0, \dots, N/2 - 1\}$$

- We want the **row-major** indexing:

$$k = (N/2)k_1 + k_2$$

- Using this index, we can write:

$$X[(N/2)k_1 + k_2] = G[k_2] + W_2^{k_1} \left( W_N^{k_2} H[k_2] \right)$$

- Not the final form yet!
- But **key take away**: We can visualize the Cooley–Tukey algorithm as arranging numbers on a matrix, performing DFTs on rows and columns!

| After Output Indexing |              |              |              |         |
|-----------------------|--------------|--------------|--------------|---------|
|                       | $k_2 = 0$    | $k_2 = 1$    | $k_2 = 2$    | $\dots$ |
| $k_1 = 0$             | $X[0]$       | $X[1]$       | $X[2]$       | $\dots$ |
| $k_1 = 1$             | $X[0 + N/2]$ | $X[1 + N/2]$ | $X[2 + N/2]$ | $\dots$ |

- The same idea extends naturally. For  $N = 3N_2$ , split  $x[n]$  into three subsequences by index modulo 3:

$$X[k] = \sum_{r=0}^{N/3-1} x[3r] W_{N/3}^{rk} + W_N^k \sum_{r=0}^{N/3-1} x[3r+1] W_{N/3}^{rk} + W_N^{2k} \sum_{r=0}^{N/3-1} x[3r+2] W_{N/3}^{rk}$$

- Define three  $N/3$ -point DFTs:

$$G_m[k] = \sum_{r=0}^{N/3-1} x[3r+m] W_{N/3}^{rk} \quad m \in \{0, 1, 2\}$$

## Radix-3 DIT Combination

$$\begin{aligned} X[k] &= G_0[k] + (W_N^k G_1[k]) + (W_N^{2k} G_2[k]) \\ X[k + N/3] &= G_0[k] + W_3^1 (W_N^k G_1[k]) + W_3^2 (W_N^{2k} G_2[k]) \\ X[k + 2N/3] &= G_0[k] + W_3^2 (W_N^k G_1[k]) + W_3^4 (W_N^{2k} G_2[k]) \end{aligned}$$

- The above 3 equations are also a 3-point DFT!

## Radix-3 DIT: Tabular Interpretation ( $N_1 = 3, N_2 = N/3$ )

- Load  $x[n]$  into a  $3 \times N/3$  matrix, **column-major**:  $n = 3n_2 + n_1$ .

- **Row 0**:  $n \equiv 0 \pmod{3}$

- **Row 1**:  $n \equiv 1 \pmod{3}$

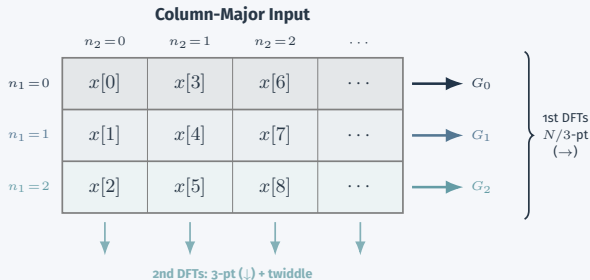
- **Row 2**:  $n \equiv 2 \pmod{3}$

1. **1st DFTs**:  $N/3$ -point DFT along each row ( $\rightarrow$ ).

2. Multiply by twiddle  $W_N^{n_1 k_2}$ .

3. **2nd DFTs**: 3-point DFT down each column ( $\downarrow$ ).

- Take output **row-major**:  
 $k = (N/3) k_1 + k_2$ .



## General Cooley–Tukey Factorization ( $N = N_1 N_2$ )

- Having seen the tabular view for Radix-2 and Radix-3, let's formalize the general  $N_1 \times N_2$  factorization.
- Re-index the input  $n$  as an  $N_1 \times N_2$  matrix in **column-major order**:

$$n = N_1 n_2 + n_1 \quad \begin{cases} n_1 \in \{0, 1, \dots, N_1 - 1\} \\ n_2 \in \{0, 1, \dots, N_2 - 1\} \end{cases}$$

Substituting this into the DFT definition yields a 2D sum:

$$X[k] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[N_1 n_2 + n_1] W_N^{(N_1 n_2 + n_1)k}$$

- Re-index the output  $k$  as an  $N_1 \times N_2$  matrix in **row-major order**:

$$k = N_2 k_1 + k_2 \quad \begin{cases} k_1 \in \{0, 1, \dots, N_1 - 1\} \\ k_2 \in \{0, 1, \dots, N_2 - 1\} \end{cases}$$

## General Cooley–Tukey Factorization ( $N = N_1 N_2$ )

- Substitute  $k = N_2 k_1 + k_2$  into the exponent  $(N_1 n_2 + n_1)k$ :

$$\begin{aligned}(N_1 n_2 + n_1)(N_2 k_1 + k_2) &= N_1 N_2 n_2 k_1 + N_1 n_2 k_2 + N_2 n_1 k_1 + n_1 k_2 \\ &= N n_2 k_1 + N_1 n_2 k_2 + N_2 n_1 k_1 + n_1 k_2\end{aligned}$$

- The first term is a full turn:  $W_N^{N n_2 k_1} = e^{-j 2\pi n_2 k_1} = 1$ , so this cross term vanishes.
- The remaining three factors are each a root of unity of a different order:

$$W_N^{N_1 n_2 k_2} = W_{N_2}^{n_2 k_2}, \quad W_N^{N_2 n_1 k_1} = W_{N_1}^{n_1 k_1}, \quad W_N^{n_1 k_2} \text{ (stays as it is)}$$

$$\text{since } W_N^{N_1 m} = e^{-j \frac{2\pi}{N_1 N_2} N_1 m} = e^{-j \frac{2\pi}{N_2} m} = W_{N_2}^m.$$

- Hence

$$W_N^{(N_1 n_2 + n_1)(N_2 k_1 + k_2)} = W_{N_2}^{n_2 k_2} \cdot W_N^{n_1 k_2} \cdot W_{N_1}^{n_1 k_1}$$

- Substituting this back into the double sum and rearranging:

## Cooley–Tukey 2D Matrix Formulation

$$X[N_2 k_1 + k_2] = \sum_{n_1=0}^{N_1-1} \underbrace{W_N^{n_1 k_2}}_{\text{twiddle}} \underbrace{\left( \sum_{n_2=0}^{N_2-1} x[N_1 n_2 + n_1] W_{N_2}^{n_2 k_2} \right)}_{N_2\text{-point DFT of row } n_1} W_{N_1}^{n_1 k_1}$$

- **Inner sum:** an  $N_2$ -point DFT along each row of the  $N_1 \times N_2$  input matrix (over  $n_2$ , indexed by  $k_2$ ).
- **Twiddle factors:** the entry at  $(n_1, k_2)$  is scaled by  $W_N^{n_1 k_2}$ .
- **Outer sum:** an  $N_1$ -point DFT down each column of the intermediate matrix (over  $n_1$ , indexed by  $k_1$ ).
- Read the result row-major:  $k = N_2 k_1 + k_2$ .

- The Cooley–Tukey matrix formulation directly yields **Bailey's FFT** (a.k.a. the Four-Step FFT):

### Bailey's FFT

1. Arrange input  $x[n]$  in an  $N_1 \times N_2$  matrix (column-major).
2. Compute  $N_2$ -point DFT of each row ( $\rightarrow$ ).
3. Multiply entry  $(n_1, k_2)$  by twiddle factor  $W_N^{n_1 k_2}$ .
4. Compute  $N_1$ -point DFT of each column ( $\downarrow$ ).

- How to compute the  $N_1$ - and  $N_2$ -point DFTs? **Apply the same algorithm recursively!**
- If  $N = p_1^{a_1} p_2^{a_2} \cdots$ , we recursively factor until only small primes remain  $\Rightarrow O(N \log N)$  for any composite  $N$ .
- This handles powers of 2, powers of 3, mixed radices—any  $N$  with small prime factors.
- But what if  $N$  itself is a **large prime**? Hold that question: the answer needs a tool we can now build — fast convolution.

## Application: Fast Convolution

---

## The Filtering Recipe, Now With the FFT

- Lecture 2: LTI filtering is the linear convolution  $y = x * h$  ( $L$  input samples,  $M$  taps), costing  $L \cdot M$  multiplications directly.
- Multiplying DFTs gives circular convolution; zero-padding to  $N \geq L + M - 1$  parks the wrapped taps on zeros, so circular = linear.
- The transform was the bottleneck:  $N^2$  per DFT. Replace every DFT by an FFT.

### Fast Convolution (input length $L$ , filter length $M$ )

1. Choose  $N \geq L + M - 1$  — the next power of 2, so Radix-2 applies.
  2. Zero-pad  $x$  and  $h$  to length  $N$ .
  3.  $X[k] = \text{FFT}\{x\}$  and  $H[k] = \text{FFT}\{h\}$ .
  4. Multiply bin by bin:  $Y[k] = X[k] H[k]$ .
  5.  $y[n] = \text{IFFT}\{Y\}$ ; keep the first  $L + M - 1$  samples.
- Padding beyond  $L + M - 1$  is harmless: the extra room only appends trailing zeros to  $y$ . (Contrast Lecture 3, where padding a periodic signal changed its period — here nothing is periodic, so nothing breaks.)

## Fast Convolution: Complexity Analysis

- How much faster is this really?
- **Direct Linear Convolution:** takes  $O(L \cdot M)$  operations. If  $L \approx M \approx N/2$ , naive convolution takes roughly  $O(N^2)$  steps.
- **Fast Convolution (using Radix-2 FFT):**
  - 2 forward FFTs:  $2 \times O(N \log N)$
  - 1 element-wise multiplication:  $O(N)$
  - 1 inverse FFT:  $O(N \log N)$

Total time:  $O(N \log N)$  complex multiplications.

### Real-world magnitude

Applying a 1-second reverb filter ( $L = 44,100$ ) to 1 second of audio ( $M = 44,100$ ):

- **Naive:**  $L \times M \approx 1.94 \times 10^9$  operations.
- **Fast Convolution:** padding to  $N = 131,072 = 2^{17}$ , taking  $3 \times (N \log_2 N) \approx 6.7 \times 10^6$  operations.
- Roughly a  $300\times$  speedup of purely algorithmic origin — what makes real-time, high-fidelity audio processing possible.

## FFT for Prime Lengths

---

## What if $N$ is Prime? Bluestein's Algorithm

- Cooley–Tukey yields  $O(N \log N)$  performance for powers of 2, or for any composite  $N = N_1 N_2$ . But what if  $N$  is a large prime number (e.g.,  $N = 1,000,003$ )?
- Then  $N_1 = 1$ , and Cooley–Tukey devolves right back to  $O(N^2)$ .
- **Bluestein's Algorithm** brings any DFT length to  $O(N \log N)$  by cleverly expressing the DFT as a linear convolution — which we now know how to compute fast.

### The Chirp-Z Identity

Start with the algebraic identity for squares:

$$(k - n)^2 = k^2 - 2kn + n^2 \implies kn = -\frac{(k - n)^2}{2} + \frac{k^2}{2} + \frac{n^2}{2}$$

Using this, we can substitute  $kn$  in the twiddle factor  $W_N^{kn} = W_{2N}^{2kn}$  giving:

$$W_N^{kn} = W_{2N}^{-(k-n)^2} W_{2N}^{k^2} W_{2N}^{n^2}$$

- Applying the Chirp-Z Identity to the DFT definition:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{kn} \\ &= \sum_{n=0}^{N-1} x[n] \left( W_{2N}^{-(k-n)^2} W_{2N}^{k^2} W_{2N}^{n^2} \right) \\ &= W_{2N}^{k^2} \sum_{n=0}^{N-1} \underbrace{\left( x[n] W_{2N}^{n^2} \right)}_{a[n]} \underbrace{W_{2N}^{-(k-n)^2}}_{b[k-n]} \end{aligned}$$

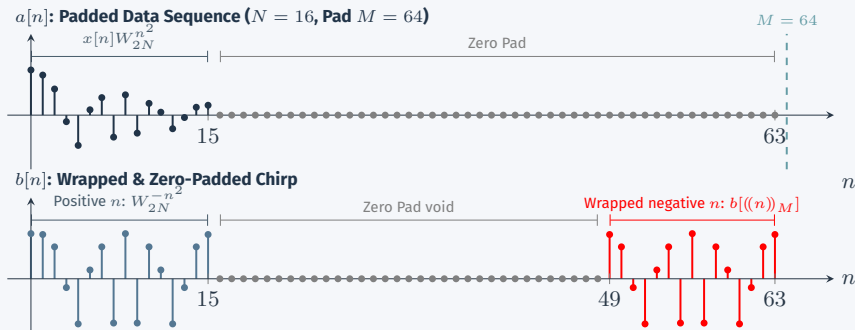
- The summation is precisely a **linear convolution** evaluated at index  $k$ !

$$X[k] = W_{2N}^{k^2} \cdot (a * b)[k]$$

where  $a[n] = x[n] W_{2N}^{n^2}$  for  $n \in [0, N-1]$  and the chirp filter  $b[n] = W_{2N}^{-n^2}$  for  $n \in [-(N-1), N-1]$ .

- We can evaluate this convolution using fast convolution!
- But in this case we need to handle the padding of  $b[n]$  with care, as  $n$  can be negative.

# Visualizing Zero-Padding for Bluestein's Algorithm



- Linear convolution evaluates  $b[k - n]$  for  $n = 0, \dots, N - 1$ , so the index  $k - n$  runs from  $k - (N - 1)$  to  $k$ : it can be negative.
- In a length- $M$  circular buffer a negative index  $n$  lands at  $((n))_M = M + n$  – Lecture 2's wrap, now used on purpose.
- So the **positive indices** of  $b[n]$  go at the start of the size- $M$  array, the **negative indices** at the end, and zeros in between so the two never overlap:  $M \geq 2N - 1$ .

- How does turning a multiplication into a convolution help?
- Because we can compute a linear convolution with **fast convolution**: zero-pad, FFT, multiply, IFFT.
- Sequence  $a[n]$  has length  $N$ . Sequence  $b[n]$  is needed for  $n \in -(N-1) \dots (N-1)$ , so its support length is  $2N-1$ .
- We can pick any  $M \geq 2N-1$  and zero-pad both sequences. We intentionally choose  $M = 2^m$  so we can use the highly efficient Radix-2 DIT FFT — the prime  $N$  never touches the transform length.

### Bluestein's FFT Algorithm

1. Form  $a[n] = x[n]W_{2N}^{n^2}$  and the wrapped chirp  $b[(n)_M]$ , both of size  $M = 2^{\lceil \log_2(2N-1) \rceil}$ .
2. Compute  $A = \text{FFT}(a, M)$  and  $B = \text{FFT}(b, M)$  using Radix-2.
3. Compute  $Y[k] = A[k]B[k]$ .
4. Compute  $y[n] = \text{IFFT}(Y, M)$ .
5. Multiply by  $W_{2N}^{k^2}$ :  $X[k] = y[k]W_{2N}^{k^2}$  for  $k = 0 \dots N-1$ .

## The Big Picture

---

### FFT for any composite $N = N_1 N_2$

- Load  $x[n]$  column-major into  $N_1 \times N_2$ ; DFT the rows; twiddle by  $W_N^{n_1 k_2}$ ; DFT the columns; read row-major.
- Radix-2 and Radix-3 are the  $N_1 = 2, 3$  cases. Recurse on the factors:  $O(N \log N)$  whenever the prime factors are small.

### FFT for prime $N$ : Bluestein

$kn = \frac{1}{2} (k^2 + n^2 - (k - n)^2)$  turns the DFT into a linear convolution with the chirp  $W_{2N}^{-n^2}$ ; run it as a padded fast convolution of any convenient length  $M \geq 2N - 1$ .

### Fast convolution

Pad to  $N \geq L + M - 1 \rightarrow$  FFT both  $\rightarrow$  multiply  $\rightarrow$  IFFT.  $O(N \log N)$  against  $O(LM)$ : about  $300\times$  for one second of audio.

### Today in one sentence

Every  $N$  has an  $O(N \log N)$  transform — factor it if you can, chirp it if you cannot — and with it, the fastest known way to filter.